Reinforcement Learning for Block Blast

From Deep Q-Networks to Deep Value Networks

Outline

1. The Game: Block Blast
2. Reward Shaping
3. DQN-Based Approaches
4. Deep Value Network (DVN)
5. Results
6. Discussion & Future Work

Outline

1. The Game: Block Blast
2. Reward Shaping
3. DQN-Based Approaches
4. Deep Value Network (DVN)
5. Results
6. Discussion & Future Work

Block Blast: Overview

Game Rules

- 8×8 binary grid (0 = empty, 1 = filled)
- Place Tetromino-like pieces on the grid
- **Clearing**: any full row *or* column is cleared
- **Game over**: no valid placements remain

RL Challenges

- Long-horizon planning
- Large action space (≤ 64 / step)
- Sparse rewards
- Combinatorial structure (3P)

7 Piece Shapes

Dot, 2×2, 3×3, Line-3, Line-4, L-small, L-large
Each randomly rotated (0°, 90°, 180°, 270°)

Two Variants

- **1P**: one piece at a time
- **3P**: three pieces per round, free order

Environment: Single-Piece Variant (1P)

Observation (Dict)

- `board`: 8×8 int grid
- `piece`: 5×5 padded shape
- `valid_placements`: 8×8 mask
- `placements_result`: *precomputed* $(8, 8, 8, 8)$ afterstate tensor + reward map

Action Space

Discrete(64) — action $a = 8r + c$
places piece top-left at (r, c)

Key Feature: Afterstates

The environment **precomputes** the resulting board for every valid placement before the agent acts.

$$\texttt{placements_result}[r, c] = (s'_{r,c},\ r_{r,c})$$

This is the core enabler of the DVN approach.

Environment: Three-Piece Variant (3P)

3P Rules

- 3 pieces presented per **round**
- Agent plays all 3 in **any chosen order**
- `pieces_used`: binary vector tracking played pieces
- `combo`: counter accumulating cleared lines across placements

Action Space

Discrete(192) = piece_idx $\times 64 + r \times 8 + c$

3P Lookahead API

`get_t_plus_3_candidates(`γ`)`

- Enumerates all $3! = 6$ orderings
- For each: all valid positions per piece
- Returns *terminal board*, *cumulative reward*, *action sequence*

Used by the DVN 3P planner (see Section 4).

Outline

1. The Game: Block Blast
2. Reward Shaping
3. DQN-Based Approaches
4. Deep Value Network (DVN)
5. Results
6. Discussion & Future Work

Reward Shaping: First Attempt

Quadratic Combo Reward

$$r = b \cdot n_{\text{cleared}}^2, \quad b = 10$$

Intuition: strongly incentivise clearing multiple lines at once.

3P Variant (kept)

$$r = b \cdot (n_{\text{cleared}}^2 + \text{combo})$$

The `combo` counter rewards sustained clearing streaks across rounds.

Why It Failed for 1P

- Clearing ≥ 2 lines at once is **rare early in training**
- Most steps give $r = 0$: **extremely sparse signal**
- Agent cannot learn what placement is "good" without feedback
- Training diverges or stalls

Reward Shaping: Final Design (1P)

Final Reward Function

$$r = \underbrace{r_{\text{survive}}}_{5.0} + \underbrace{b \cdot n_{\text{cleared}}}_{10\times\#\text{cleared lines}} + \underbrace{b \cdot \rho}_{\text{emptiness bonus}} - \underbrace{p_{\text{no3x3}}}_{20 \text{ if no free } 3\times3}$$

Components

- $r_{\text{survive}} = 5$: reward for not dying
- n_{cleared}: rows + columns cleared
- $\rho \in [0, 1]$: fraction of **empty cells** — keeps board clean
- $p_{\text{no3x3}} = 20$: penalty if no 3×3 empty window exists ("too full")

Invalid Move

Penalty -100, **episode terminates**.

Design Philosophy

Dense signal at *every* step, not just when lines clear. The emptiness ratio teaches the agent to **preserve space** for future pieces.

Outline

1. The Game: Block Blast
2. Reward Shaping
3. DQN-Based Approaches
4. Deep Value Network (DVN)
5. Results
6. Discussion & Future Work

DDQN and Rainbow: Architecture

`BlockBlastCNNNet1P` Backbone

Layer	Output
Board + mask $(2, 8, 8)$	
Conv2d$(2 \to 32,\ k = 3)$	$32 \times 8 \times 8$
Conv2d$(32 \to 64,\ k = 3)$	$64 \times 8 \times 8$
Flatten	4096
Piece $(1, 5, 5)$	
Conv2d$(1 \to 32,\ k = 3)$	$32 \times 5 \times 5$
Flatten	800
Concat	4896
Linear $4896 \to 512 \to 256 \to 64$	64 Q-values

2,674,464 parameters

Rainbow Extensions

1. Distributional RL (C51, 51 atoms)
2. Prioritized Replay (PER)
3. N-step returns $(n = 3)$
4. Noisy Networks
5. Dueling Architecture

DQN: Why It Struggled

Root Causes of Poor Performance

1. **Credit assignment**: a good placement may not clear a line for 3+ more steps. With $\gamma = 0.99$, the discounted signal is weak over long horizons.
2. **Q-function instability**: 64 action outputs, highly variable valid masks step-to-step. Q-values must track a non-stationary landscape.
3. **Sample efficiency**: 2.67M parameters, only 1,000 training episodes. The model is severely underfit.
4. **Perception overhead**: the network must simultaneously learn to recognise piece shapes, valid positions, *and* long-term board value. This is a hard joint problem.

Key Insight

DQN treats this as a raw perception problem. We can do better by **leveraging the environment structure**.

Outline

1. The Game: Block Blast
2. Reward Shaping
3. DQN-Based Approaches
4. Deep Value Network (DVN)
5. Results
6. Discussion & Future Work

DVN: Afterstate Formulation

Core Idea

Instead of learning $Q(s, a)$, learn a **state value function**:

$$V_\theta(s') \approx \text{long-term value of board } s'$$

where s' is the board *after* placement and line clearing.

Action Selection (1-step lookahead)

$$a^* = \arg\max_a \left[r_a + \gamma \cdot V_\theta(s'_a)\right]$$

Enumerate all valid afterstates $\{s'_a, r_a\}$ via `placements_result`, batch-evaluate V_θ, pick best.

Advantages over DQN

- Network only evaluates **boards**, not (board, action) pairs
- Valid constraint handled implicitly
- Value captures board quality **independently of current piece**
- Much simpler function to learn

Key Requirement

Environment must expose precomputed afterstates. Our `placements_result` does exactly this.

DVN Architecture: Multi-Kernel CNN

8 Parallel Branches on 8×8 Board

Kernel	Role	Filters	Pad
1×1	Per-cell occupancy	1	0
2×2	Small block patterns	8	1
3×3	Piece-size patterns	16	1
4×4	Medium structures	16	2
5×5	Larger structures	16	2
8×8	Global board summary	64	0
1×8	Row detector	4	0
8×1	Column detector	4	0

Wall Padding

Padding uses `ConstantPad2d` with value **1.0**: the kernel sees "filled wall cells" outside the grid boundary.

No padding for line kernels (1×8, 8×1).

Branch Output

Each branch: Conv $\rightarrow$ LeakyReLU $\rightarrow$ Flatten $\rightarrow$ Linear$(\cdot \rightarrow 16)$ $\rightarrow$ Linear$(16 \rightarrow 8)$

Concat: $8 \times 8 = \mathbf{64}$ features

DVN Architecture: MLP Head & Parameter Count

MLP Head

$$64 \xrightarrow{\text{Linear}} 64 \xrightarrow{\text{LReLU}} 16 \xrightarrow{\text{LReLU}} 1$$

Scalar output: $V_\theta(s')$

Total: **42,461 parameters**

$63\times$ fewer than DDQN/Rainbow!

Model Comparison

Model	Params
DDQN 1P	2,674,464
Rainbow 1P	2,674,464
DDQN 3P	2,710,560
DVN (ours)	**42,461**

Why so small?

The network only needs to answer: *"Is this board position good for the long term?"* It does not need to reason about pieces, action encodings, or valid masks.

DVN: Training

TD Learning on Afterstates

Target (non-terminal):

$$y_t = \max_{a'} \left[r_{t+1,a'} + \gamma \cdot V_{\theta^-}(s'_{t+1,a'}) \right]$$

Target (terminal):

$$y_t = \frac{p_{\text{invalid}}}{\gamma}$$

Loss: Huber (SmoothL1) + grad clip ≤ 10

Training Protocol

- 10,000 episodes, max 100 steps
- ε: $1.0 \to 0.01$, decay $\times 0.999$/ep
- Buffer: 100,000 transitions
- Batch size: 128
- Adam, lr $= 10^{-4}$, StepLR
- Target net update: every 200 ep
- Logged via **Weights & Biases**
- Checkpoints every 500 ep (full state)

DVN: 3P Planner

`RoundPlanner3P`

At each new round:

1. Call `get_t_plus_3_candidates(`γ`)`
2. Score each candidate sequence:

$$\text{score} = R_{t+1:t+3} + \gamma^3 \cdot V_\theta(s_{t+3})$$

3. Select **best 3-action sequence**
4. Execute one action at a time
5. **Replan** if an action becomes invalid

Computational Cost

Full enumeration: $3! \times 64^3$ candidates in the worst case.

Pure Python/NumPy: **very slow** for online evaluation.

Future work: batched GPU lookahead.

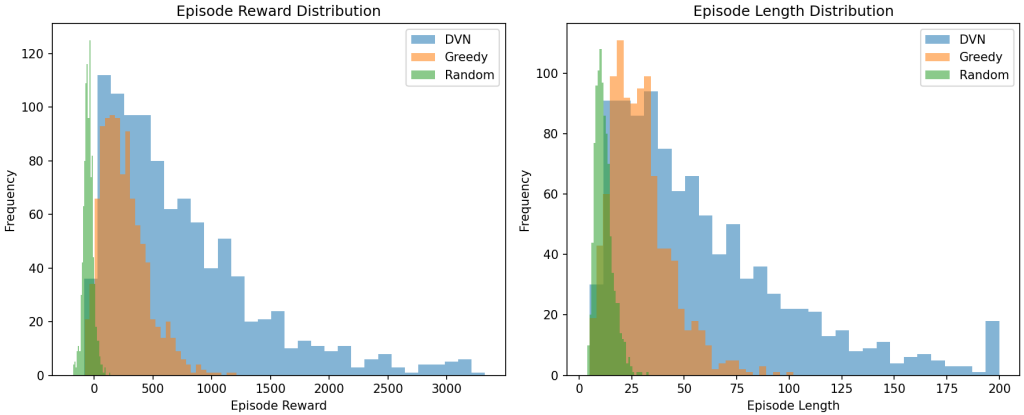
Key Feature

The DVN value function is **shared** between 1P and 3P — the planner is just a wrapper around the same V_θ.

Outline

1. The Game: Block Blast
2. Reward Shaping
3. DQN-Based Approaches
4. Deep Value Network (DVN)
5. Results
6. Discussion & Future Work

Benchmark: DVN vs. Greedy vs. Random



Episode return (left) and episode length (right) distributions over 200 evaluation episodes.

Results Summary

Observations

- **Trained DVN**: ∼60 avg steps per episode
- **Untrained DVN**: ∼20 steps (random play)
- **Greedy baseline**: picks highest immediate reward — shortsighted, over-fills board
- **DVN** outperforms greedy in episode length by learning to preserve board space

Agent	Avg Steps	Params
Random	∼5	—
Greedy	∼30	—
DVN (untrained)	∼20	42K
DVN (trained)	**∼ 60**	**42K**
DDQN 1P	∼20	2.67M
Rainbow	∼20	2.67M

Takeaway

The DVN learns real long-term value with $63\times$ fewer parameters than DQN-based approaches, by leveraging environment structure (precomputed afterstates).

Outline

1. The Game: Block Blast
2. Reward Shaping
3. DQN-Based Approaches
4. Deep Value Network (DVN)
5. Results
6. Discussion & Future Work

Discussion

Why DVN > DQN here

- Decouples *"what does this board look like?"* from *"which action achieves it?"*
- No need to reason about piece geometry in the value net
- Simpler learning objective = more sample-efficient
- Valid constraints automatically respected

Current Limitations

- Value function is **piece-agnostic**: a board's value depends on what pieces come next
- 3P planner is **computationally expensive** (exhaustive enumeration)
- Training still requires many episodes; no curriculum

Future Work

Immediate Improvements

- **Include upcoming piece info**: encode next piece(s) as extra input channels in V_θ
- **Hand-crafted feature branch**: holes count, row/column fill density, height variance — well-studied in Tetris literature
- **Batch 3P lookahead on GPU**: use tensor operations over all $3! \times 64^3$ candidates for tractable online evaluation

Longer-Term Directions

- **MCTS + DVN**: use the value function as a rollout evaluator in Monte Carlo Tree Search for deeper planning
- **Self-play curriculum**: progressively harder piece distributions
- **Model-based RL**: learn a transition model from afterstates to plan deeper than 1-step

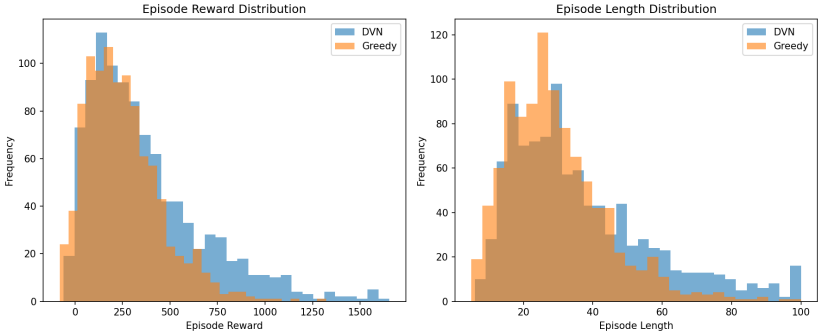
Conclusion

Summary

1. Designed and implemented **two Block Blast environments** (1P and 3P) with Gymnasium, including precomputed afterstates
2. Iterated on **reward shaping**: quadratic combo reward was too sparse; dense survival + emptiness reward works better
3. **DDQN and Rainbow DQN** ($\sim$2.67M params) failed to converge meaningfully in 1,000 episodes due to sparse rewards and large Q-function complexity
4. **Deep Value Network** (42,461 params, multi-kernel CNN): learns $V(\text{afterstate})$ with TD learning, combined with 1-step lookahead for action selection
5. DVN achieves $\sim$60 average episode steps vs. $\sim$30 for greedy, with $63\times$ fewer parameters

Leveraging environment structure is more powerful than bigger models.

Appendix: Benchmark (Earlier Run)



Earlier benchmark comparing DVN vs. greedy only (before random baseline was added).

Appendix: DVN Training Algorithm

Algorithm (simplified)

Input: V_θ, target V_{θ^-}, env, buffer $\mathcal{B}$

For each episode:
  For each step t:
    Compute all afterstates $\{(s'_a, r_a)\}$
    $a^* = \arg\max_a [r_a + \gamma V_\theta(s'_a)]$ (greedy)
    or sample random (exploration)
    Store $(s'_{a^*}, r_{a^*}, s_{\text{next}}, \text{done})$ in $\mathcal{B}$
    Sample batch from $\mathcal{B}$
    Compute TD targets y_t
    Update θ via Huber loss
  Periodically: $\theta^- \leftarrow \theta$

Hyperparameters

Param	Value
γ	0.99
$\varepsilon_0 \to \varepsilon_{\min}$	$1.0 \to 0.01$
Buffer size	100,000
Batch size	128
LR	10^{-4}
Target update	200 ep